

Optimization

EMEN 7029 & EMEN 7104 - Special Topic In
Electromechanical Engineering / Robotics

Qingbiao Li

University of Macau

Optimization	2
Calculus and Critical Points	8
Multivariate Calculus	16
Deriving Linear Regression	25
Gradient Descent	40
Backpropagation	54
Coding	74

Optimization

Optimization

We previously introduced neural networks as universal function approximators

A deep neural network f can approximate **any** continuous function g to infinite precision

$$f(x, \theta) = g(x) + \varepsilon; \quad \varepsilon \rightarrow 0$$

g can be a mapping from pictures to text, English to Chinese, etc

But how do we find the θ that makes $f(x, \theta) = g(x) + \varepsilon$?

We said this θ exists, but never said how to find it

Goal: Find the parameters θ a neural network

Optimization

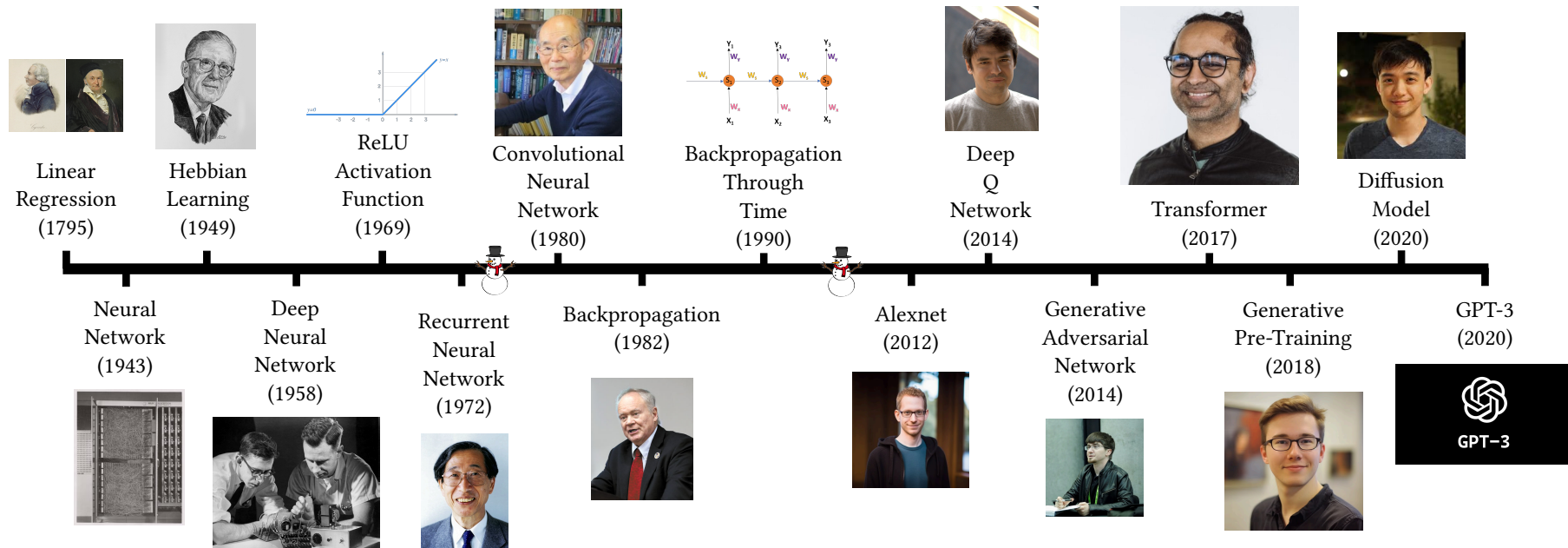
When we search for θ , we call it **optimization** or **training**

We optimize a loss function by computing

$$\arg \min_{\theta} \mathcal{L}(\mathbf{X}, \mathbf{Y}, \theta)$$

This expression looks very simple, but it can be very hard to evaluate

Optimization



Neural networks were discovered in 1943, but we could not train them until 1982!

This is why theory is important

Optimization

Recall how we found θ in the linear regression problem

We define the square error loss function

$$\arg \min_{\theta} \mathcal{L}(\mathbf{X}, \mathbf{Y}, \theta) = \arg \min_{\theta} \sum_{i=1}^n \left(f(\mathbf{x}_{[i]}, \theta) - \mathbf{y}_{[i]} \right)^2$$

Then, I gave you a magical solution to this optimization problem

$$\theta = \left(\overline{\mathbf{X}}^{\top} \overline{\mathbf{X}} \right)^{-1} \overline{\mathbf{X}}^{\top} \mathbf{Y}$$

Where does this solution come from? Can we do the same for neural networks?

Optimization

The solution for linear regression and neural networks comes from **calculus**

The solution for neural networks also comes from calculus

Let us review basic calculus concepts

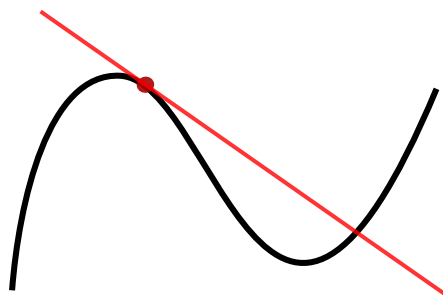
Calculus and Critical Points

Calculus and Critical Points

We write the **derivative** of a function f with respect to an input x as

$$f'(x) = \frac{d}{dx} f = \frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

The derivative is the slope of a function



$$f(x), f'(a)$$

Calculus and Critical Points

It is easiest if you treat the derivative as a **function of functions**

$$\text{derivative}(f) = f'$$

$$\frac{d}{dx} : f \mapsto f'$$

The derivative takes a function f and outputs a new function f'

$$\frac{d}{dx} : (f : X \mapsto Y) \mapsto (f' : X \mapsto \mathbb{R})$$

Calculus and Critical Points

There are formulas for computing the derivative of various operations

Constant

$$\frac{d}{dx}c = 0$$

Power

$$\frac{d}{dx}x^n = nx^{n-1}$$

Calculus and Critical Points

Sum/Difference

$$\frac{d}{dx}(f(x) + g(x)) = f'(x) + g'(x)$$

Product

$$\frac{d}{dx}(f(x)g(x)) = f'(x)g(x) + f(x)g'(x)$$

Chain

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$$

Calculus and Critical Points

For example, consider the function

$$f(x) = x^2 - 3x$$

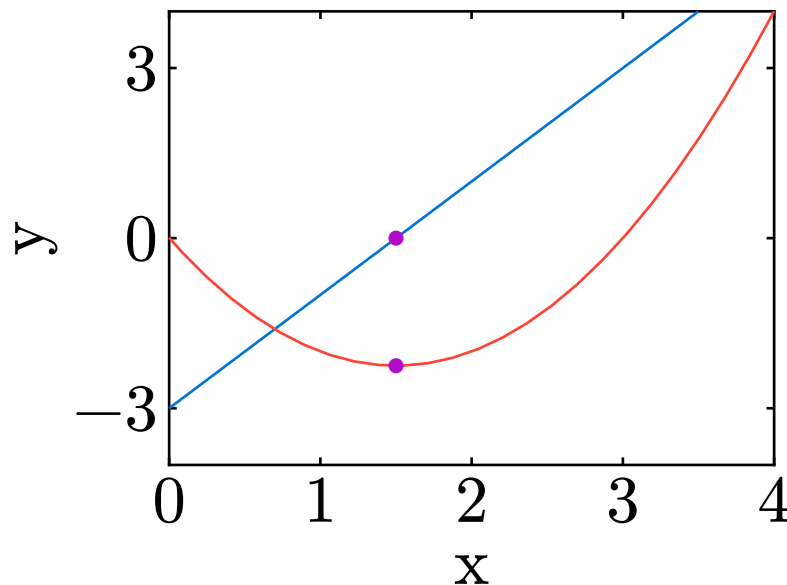
We can write the derivative as

$$f'(x) = 2x - 3$$

We can evaluate the derivative at specific points

$$f'(1) = 2 \cdot 1 - 3 = -1$$

Calculus and Critical Points



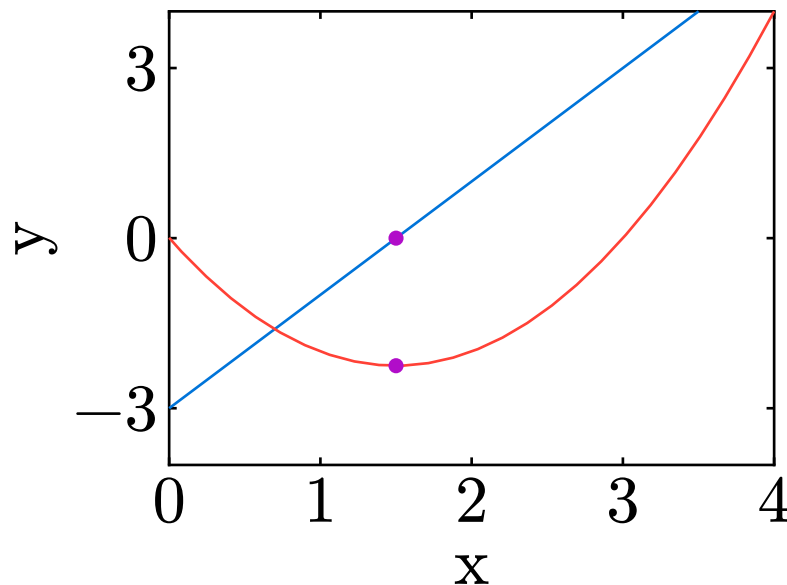
$$f(x) = x^2 - 3x$$

$$f'(x) = 2x - 3$$

Question: What happens to $f(x)$ at $f'(x) = 0$?

Answer: Critical point, local minima or local maxima

Calculus and Critical Points



$$f(x) = x^2 - 3x$$

$$f'(x) = 2x - 3$$

Question: Find critical points?

$$0 = 2x - 3 \Rightarrow x = \frac{3}{2}$$

Question: Why do we care about critical points in deep learning?

Answer: Optimization finds minima, always at critical points

Multivariate Calculus

Multivariate Calculus

Standard calculus notation is for functions of one variable

Question: Are neural networks functions of one variable? **Answer:** No

For neural networks, we need **multivariate calculus** (vector calculus)

The ideas are exactly the same, but become more complicated

Question: What does $f'(x, y, z)$ mean?

Answer: Unknown, need new notation for multivariate calculus

Multivariate Calculus

We introduce the **gradient operator**

$$\nabla : \underbrace{(f : \mathbb{R}^n \mapsto \mathbb{R})}_{\text{Vector function}} \times \underbrace{\mathbb{Z}_+^m}_{\text{Differentiation variables}} \mapsto \underbrace{(f' : \mathbb{R}^n \mapsto \mathbb{R}^m)}_{\text{Slope at differentiation variables}}$$

$$\mathbf{y} = \nabla_{\mathbf{x}} f(x)$$

Multivariate Calculus

For one dimension, the gradient is the same as the derivative

$$\begin{array}{c} \text{Inputs to } f' \\ (\nabla_x f)(x) = \frac{\partial f}{\partial x}(x) \\ \text{New function } f' \end{array}$$

You can omit the differentiation variable as there is only one choice

$$(\nabla f)(x)$$

Multivariate Calculus

The gradient generalizes the derivative to vector functions

$$\left(\nabla_{x_1, x_3} f \right) \left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \right) = \begin{bmatrix} \frac{\partial}{\partial x_1} f \left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \right) \\ \frac{\partial}{\partial x_3} f \left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \right) \end{bmatrix}$$

$$(\nabla_{\mathbf{x}} f)(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_{d_x}} \end{bmatrix}$$

Multivariate Calculus

We will even see multivariate vector functions

Question: How do I write the following?

$$(\nabla_{\boldsymbol{\theta}} f)(\boldsymbol{x}, \boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial f(\boldsymbol{x}, \boldsymbol{\theta})}{\partial \theta_0} \\ \frac{\partial f(\boldsymbol{x}, \boldsymbol{\theta})}{\partial \theta_1} \\ \vdots \\ \frac{\partial f(\boldsymbol{x}, \boldsymbol{\theta})}{\partial \theta_{d_x}} \end{bmatrix}$$

Multivariate Calculus

Example: Consider the function

$$f(x_1, x_2) = x_1^2 - 3x_1x_2$$

We can write the gradient as

$$(\nabla_{\mathbf{x}} f)(\mathbf{x}) = (\nabla_{x_1, x_2} f) \left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = \begin{bmatrix} \frac{\partial}{\partial x_1} f(x_1, x_2) \\ \frac{\partial}{\partial x_2} f(x_1, x_2) \end{bmatrix} = \begin{bmatrix} 2x_1 - 3x_2 \\ -3x_1 \end{bmatrix}$$

Multivariate Calculus

$$(\nabla_{\mathbf{x}} f)(\mathbf{x}) = (\nabla_{x_1, x_2} f) \left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = \begin{bmatrix} \frac{\partial}{\partial x_1} f(x_1, x_2) \\ \frac{\partial}{\partial x_2} f(x_1, x_2) \end{bmatrix} = \begin{bmatrix} 2x_1 - 3x_2 \\ -3x_1 \end{bmatrix}$$

Example: Evaluate the gradient at specific points ($x_1 = 1, x_2 = 0$)

$$(\nabla_{\mathbf{x}} f) \left(\begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) = (\nabla_{x_1, x_2} f) \left(\begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} \frac{\partial}{\partial x_1} f(1, 0) \\ \frac{\partial}{\partial x_2} f(1, 0) \end{bmatrix} = \begin{bmatrix} 2 \cdot 1 - 3 \cdot 0 \\ -3 \cdot 1 \end{bmatrix} = \begin{bmatrix} -1 \\ -3 \end{bmatrix}$$

Multivariate Calculus

Remember, critical points (minima) at $f'(x) = 0$

With a multivariate function, the critical points at $\nabla_x f(\mathbf{x}) = 0$

$$(\nabla_{\mathbf{x}} f)(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_n} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Deriving Linear Regression

Deriving Linear Regression

Now that we remember calculus, let us revisit linear regression

We will use vector calculus to derive the solution for linear regression

We will use this solution to derive the solution for deep neural networks

Deriving Linear Regression

In linear regression, our loss function is

$$\mathcal{L}(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = \sum_{i=1}^n \left(f(\mathbf{x}_{[i]}, \boldsymbol{\theta}) - \mathbf{y}_{[i]} \right)^2$$

We can write the square error loss in matrix form as

$$\mathcal{L}(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = (\mathbf{Y} - \bar{\mathbf{X}}\boldsymbol{\theta})^\top (\mathbf{Y} - \bar{\mathbf{X}}\boldsymbol{\theta})$$

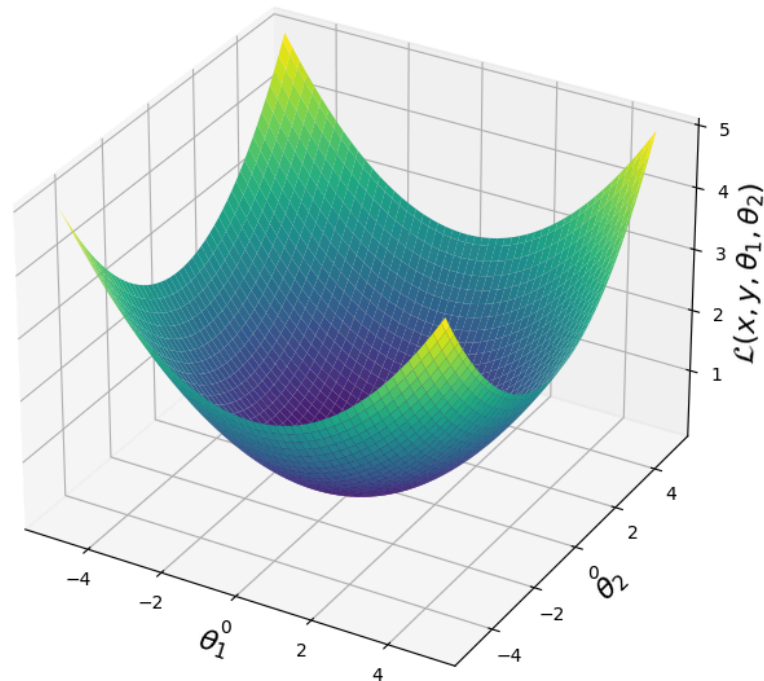
$$\mathcal{L}(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = \underbrace{\underbrace{(\mathbf{Y} - \bar{\mathbf{X}}\boldsymbol{\theta})^\top}_{\text{Linear function of } \boldsymbol{\theta}} \underbrace{(\mathbf{Y} - \bar{\mathbf{X}}\boldsymbol{\theta})}_{\text{Linear function of } \boldsymbol{\theta}}}_{\text{Quadratic function of } \boldsymbol{\theta}}$$

Deriving Linear Regression

$$\mathcal{L}(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = \underbrace{\left(\mathbf{Y} - \overline{\mathbf{X}}\boldsymbol{\theta}\right)^\top}_{\text{Linear function of } \boldsymbol{\theta}} \underbrace{\left(\mathbf{Y} - \overline{\mathbf{X}}\boldsymbol{\theta}\right)}_{\text{Quadratic function of } \boldsymbol{\theta}}$$

A quadratic function has a single minima! The minima must be at

$$(\nabla_{\boldsymbol{\theta}}\mathcal{L})(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = 0$$



Deriving Linear Regression

Therefore, we know that the θ that solves

$$(\nabla_{\theta} \mathcal{L})(\mathbf{X}, \mathbf{Y}, \theta) = 0$$

Also solves

$$\arg \min_{\theta} \mathcal{L}(\mathbf{X}, \mathbf{Y}, \theta)$$

Deriving Linear Regression

Using calculus, let us derive the solution to linear regression

$$\mathcal{L}(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = (\mathbf{Y} - \overline{\mathbf{X}}\boldsymbol{\theta})^\top (\mathbf{Y} - \overline{\mathbf{X}}\boldsymbol{\theta})$$

$$(\nabla_{\boldsymbol{\theta}}\mathcal{L})(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \left[(\mathbf{Y} - \overline{\mathbf{X}}\boldsymbol{\theta})^\top (\mathbf{Y} - \overline{\mathbf{X}}\boldsymbol{\theta}) \right]$$

$$= \nabla_{\boldsymbol{\theta}} \left[\mathbf{Y}^\top \mathbf{Y} - \mathbf{Y}^\top \overline{\mathbf{X}}\boldsymbol{\theta} - (\overline{\mathbf{X}}\boldsymbol{\theta})^\top \mathbf{Y} + (\overline{\mathbf{X}}\boldsymbol{\theta})^\top \overline{\mathbf{X}}\boldsymbol{\theta} \right]$$

$$= \mathbf{0} - \mathbf{Y}^\top \overline{\mathbf{X}}\mathbf{I} - (\overline{\mathbf{X}}\mathbf{I})^\top \mathbf{Y} + (\overline{\mathbf{X}}\mathbf{I})^\top \overline{\mathbf{X}}\boldsymbol{\theta} + (\overline{\mathbf{X}}\boldsymbol{\theta})^\top \overline{\mathbf{X}}\mathbf{I}$$

Deriving Linear Regression

$$= \mathbf{0} - \mathbf{Y}^\top \overline{\mathbf{X}} \mathbf{I} - (\overline{\mathbf{X}} \mathbf{I})^\top \mathbf{Y} + (\overline{\mathbf{X}} \mathbf{I})^\top \overline{\mathbf{X}} \boldsymbol{\theta} + (\overline{\mathbf{X}} \boldsymbol{\theta})^\top \overline{\mathbf{X}} \mathbf{I}$$

$$= -\mathbf{Y}^\top \overline{\mathbf{X}} - \overline{\mathbf{X}}^\top \mathbf{Y} + \overline{\mathbf{X}}^\top \overline{\mathbf{X}} \boldsymbol{\theta} + (\overline{\mathbf{X}} \boldsymbol{\theta})^\top \overline{\mathbf{X}}$$

Remember, $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$, and so $\mathbf{Y}^\top \overline{\mathbf{X}} = \mathbf{Y}^\top (\overline{\mathbf{X}}^\top)^\top = \overline{\mathbf{X}}^\top \mathbf{Y}$

$$= -\mathbf{Y}^\top \overline{\mathbf{X}} - \mathbf{Y}^\top \overline{\mathbf{X}} + \overline{\mathbf{X}}^\top \overline{\mathbf{X}} \boldsymbol{\theta} + \overline{\mathbf{X}}^\top \overline{\mathbf{X}} \boldsymbol{\theta}$$

$$= -2\overline{\mathbf{X}}^\top \mathbf{Y} + 2\overline{\mathbf{X}}^\top \overline{\mathbf{X}} \boldsymbol{\theta}$$

And so the gradient of the loss is

$$(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = -2\overline{\mathbf{X}}^\top \mathbf{Y} + 2\overline{\mathbf{X}}^\top \overline{\mathbf{X}} \boldsymbol{\theta}$$

Deriving Linear Regression

$$(\nabla_{\theta}\mathcal{L})(X, Y, \theta) = -2\overline{X}^{\top}Y + 2\overline{X}^{\top}\overline{X}\theta$$

We want to find the θ that makes the gradient of the loss zero

$$0 = -2\overline{X}^{\top}Y + 2\overline{X}^{\top}\overline{X}\theta$$

$$2\overline{X}^{\top}Y = 2\overline{X}^{\top}\overline{X}\theta$$

$$\overline{X}^{\top}Y = \overline{X}^{\top}\overline{X}\theta$$

$$\left(\overline{X}^{\top}\overline{X}\right)^{-1}\overline{X}^{\top}Y = \theta$$

Deriving Linear Regression

$$\left(\overline{\mathbf{X}}^\top \overline{\mathbf{X}}\right)^{-1} \overline{\mathbf{X}}^\top \mathbf{Y} = \boldsymbol{\theta}$$

This was the “magic” solution I gave you for linear regression

$$\boldsymbol{\theta} = \left(\overline{\mathbf{X}}^\top \overline{\mathbf{X}}\right)^{-1} \overline{\mathbf{X}}^\top \mathbf{Y}$$

Deriving Linear Regression

Great! We derived the solution to linear regression

Now, we will do the same approach for neural networks

To make it simple, we assume $d_x = 1, d_y = 1, n = 1$

One input dimension, one output dimension, one datapoint

Step 1: Write the loss function for a neural network

Deriving Linear Regression

Like linear regression, we can use square error for a neural network

$$\mathcal{L}(x, y, \boldsymbol{\theta}) = (f(x, \boldsymbol{\theta}) - y)^2$$

All that changes is the model f

Linear regression:

$$f(x, y, \boldsymbol{\theta}) = \theta_0 + \theta_1 x$$

Perceptron:

$$f(x, y, \boldsymbol{\theta}) = \sigma(\theta_0 + \theta_1 x)$$

Deriving Linear Regression

$$\mathcal{L}(x, y, \theta) = (f(x, \theta) - y)^2$$

Loss function

$$f(x, \theta) = \sigma(\theta_0 + \theta_1 x)$$

Neural network model

Now, we plug the model f into the loss function

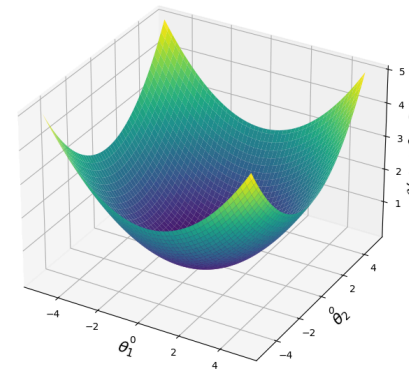
$$\mathcal{L}(x, y, \theta) = (\sigma(\theta_0 + \theta_1 x) - y)^2$$

Rewrite

$$\mathcal{L}(x, y, \theta) = \underbrace{(\sigma(\theta_0 + \theta_1 x) - y)}_{\text{Nonlinear function of } \theta} \underbrace{(\sigma(\theta_0 + \theta_1 x) - y)}_{\text{Nonlinear function of } \theta}$$

Deriving Linear Regression

Linear regression loss function
was quadratic with one minima



With a neural network, this is our loss function

$$\mathcal{L}(x, y, \theta) = \underbrace{(\sigma(\theta_0 + \theta_1 x) - y)}_{\text{Nonlinear function of } \theta} \underbrace{(\sigma(\theta_0 + \theta_1 x) - y)}_{\text{Nonlinear function of } \theta}$$

Question: How many minima does this function have?

Answer: We do not know

Deriving Linear Regression

The nonlinearity/activation function σ in the neural network means we cannot find an analytical solution

$$f(x, \boldsymbol{\theta}) = \sigma(\theta_0 + \theta_1 x)$$

Question: Can we remove the activation function σ ?

Answer: Yes, but the result is linear regression

$$f(x, \boldsymbol{\theta}) = \theta_0 + \theta_1 x$$

Activation functions make the neural network powerful

Deriving Linear Regression

Linear regression: analytical solution for θ

Neural network: no analytical solution for θ

So how to find θ for a neural network?

Gradient Descent

Gradient Descent

To find θ for a neural network, we use **gradient descent**

Gradient descent optimizes differentiable or smooth functions

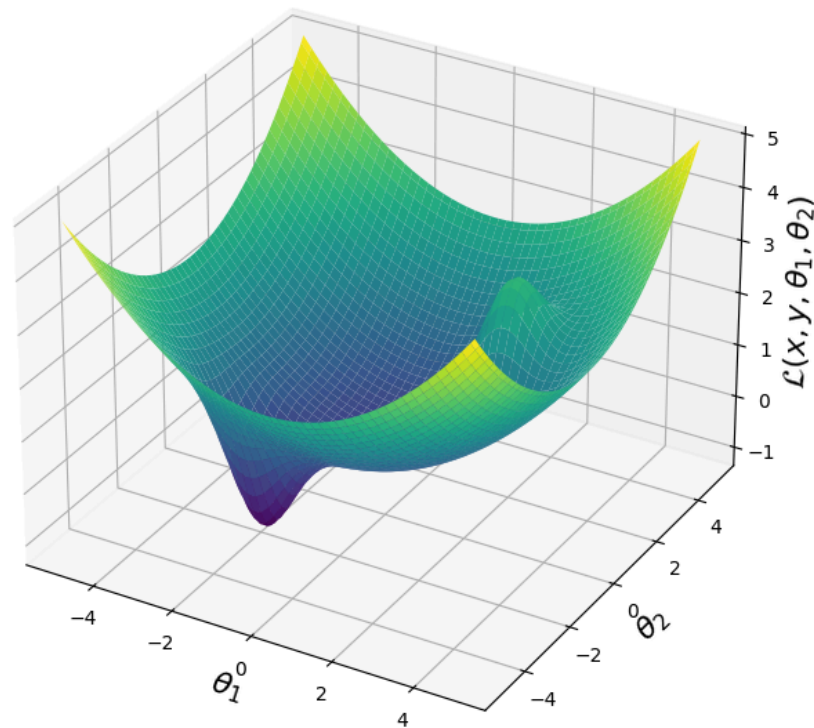
Careful, gradient is not defined for all functions!

Can only use gradient descent on differentiable/smooth loss functions

How does gradient descent work?

Gradient Descent

A differentiable loss function
produces a manifold

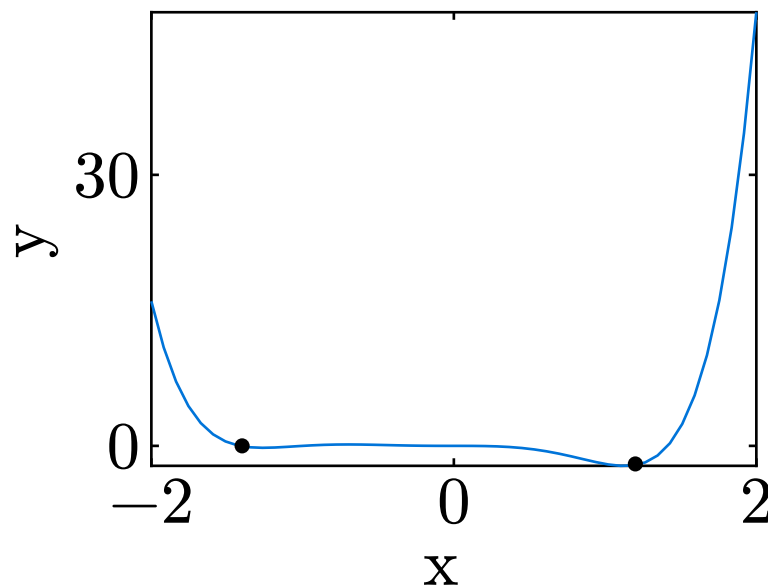


Our goal is to find the lowest point (minima) on this manifold

The lowest point solves $\arg \min_{\theta} \mathcal{L}(\mathbf{X}, \mathbf{Y}, \theta)$

Gradient Descent

Note: Gradient descent only provides **local** optima, not **global** optima



In practice, a local optima provides a good enough model

Gradient Descent

Let us define gradient descent without math

You are on the top of a mountain and there is lightning storm



For safety, you should walk down the mountain to escape the lightning

Gradient Descent

But you do not know the path down!



You see this, which way do you walk next?

Gradient Descent

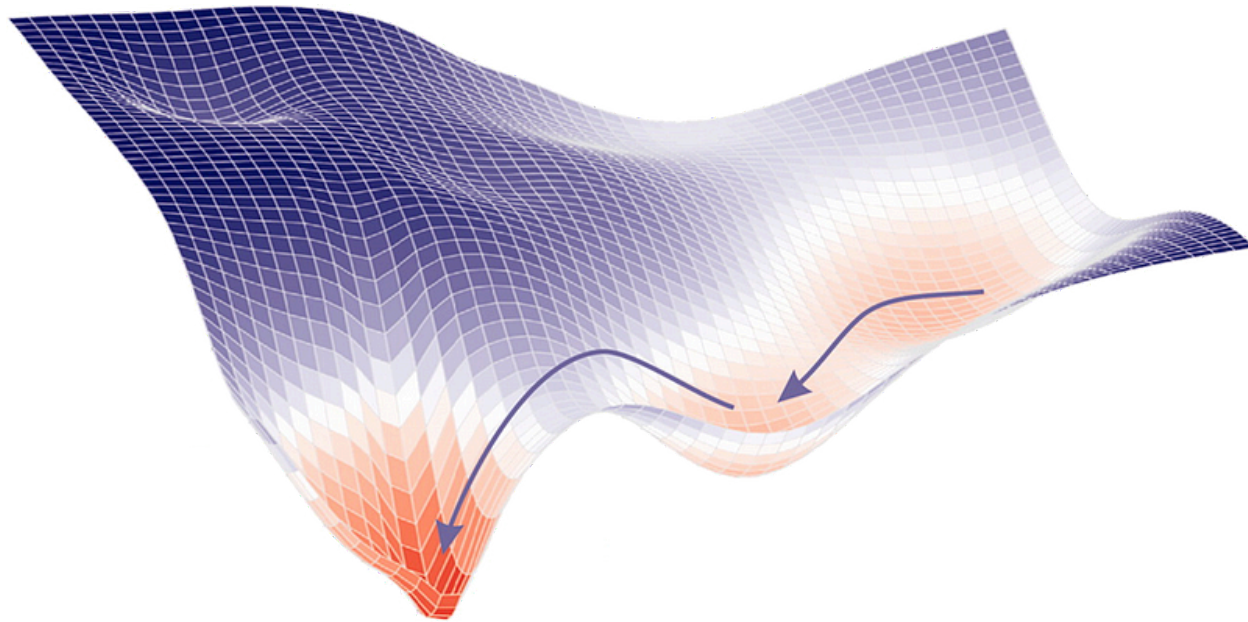


This is gradient descent

Gradient Descent

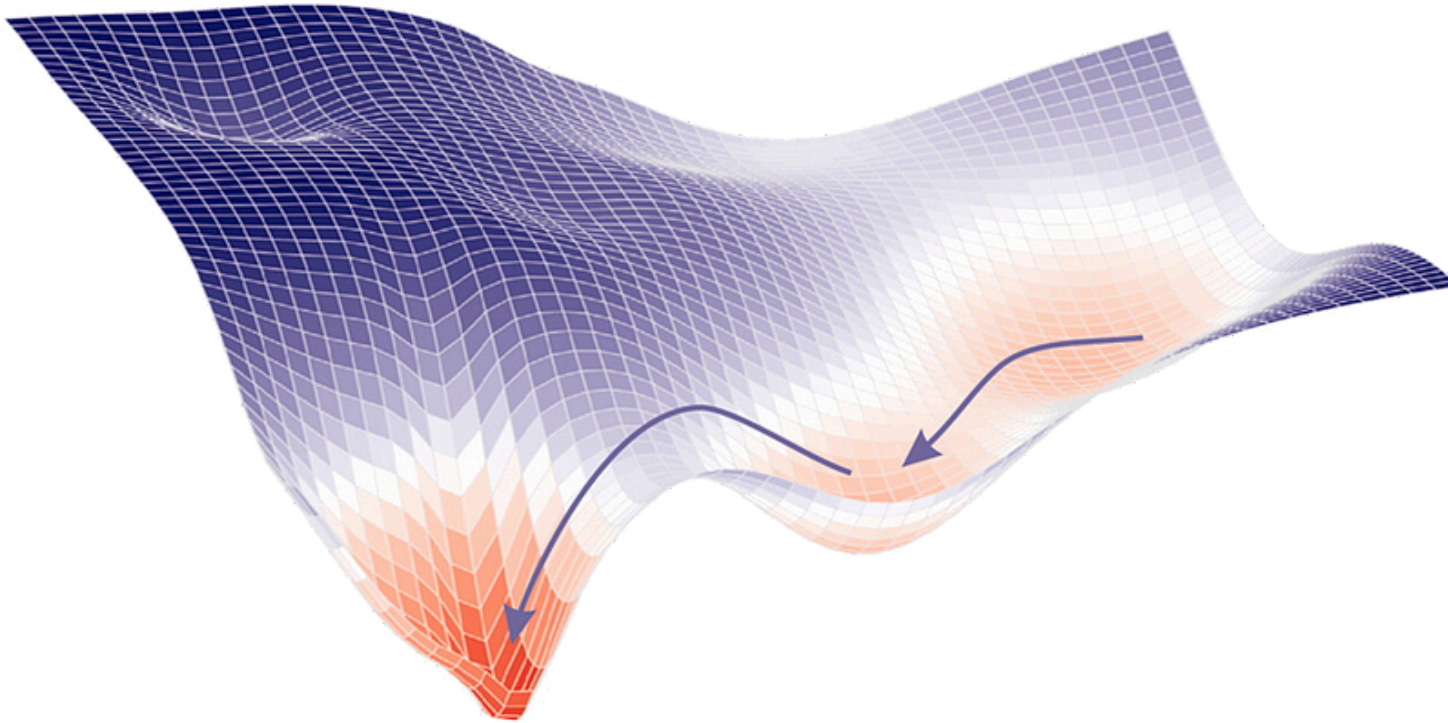
In gradient descent, we look at the **slope** (∇) of the loss function

And we walk in the steepest direction



And then we repeat

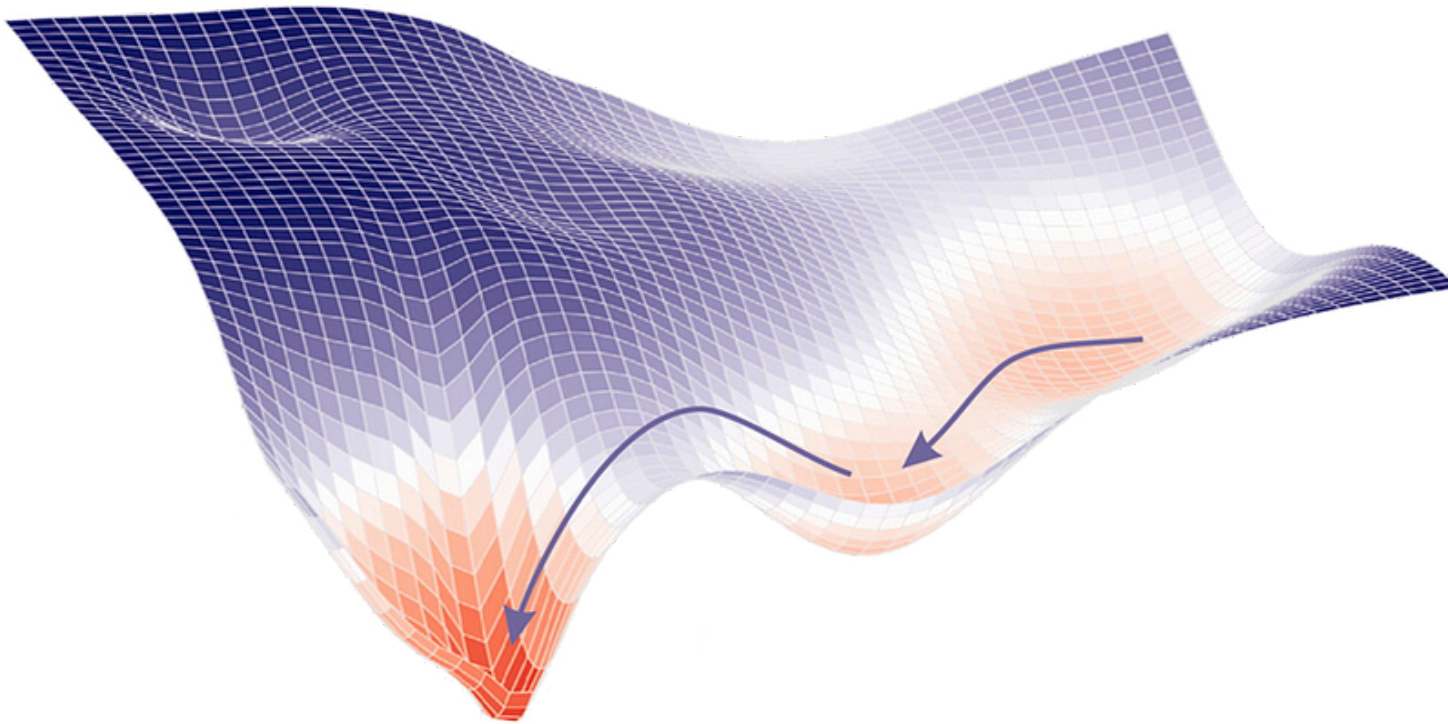
Gradient Descent



We find the gradient $(\nabla_{\theta}\mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$

And update $\boldsymbol{\theta}$ in the steepest direction

Gradient Descent



Eventually, we arrive at the bottom

Gradient Descent

With gradient descent, the loss function must be differentiable

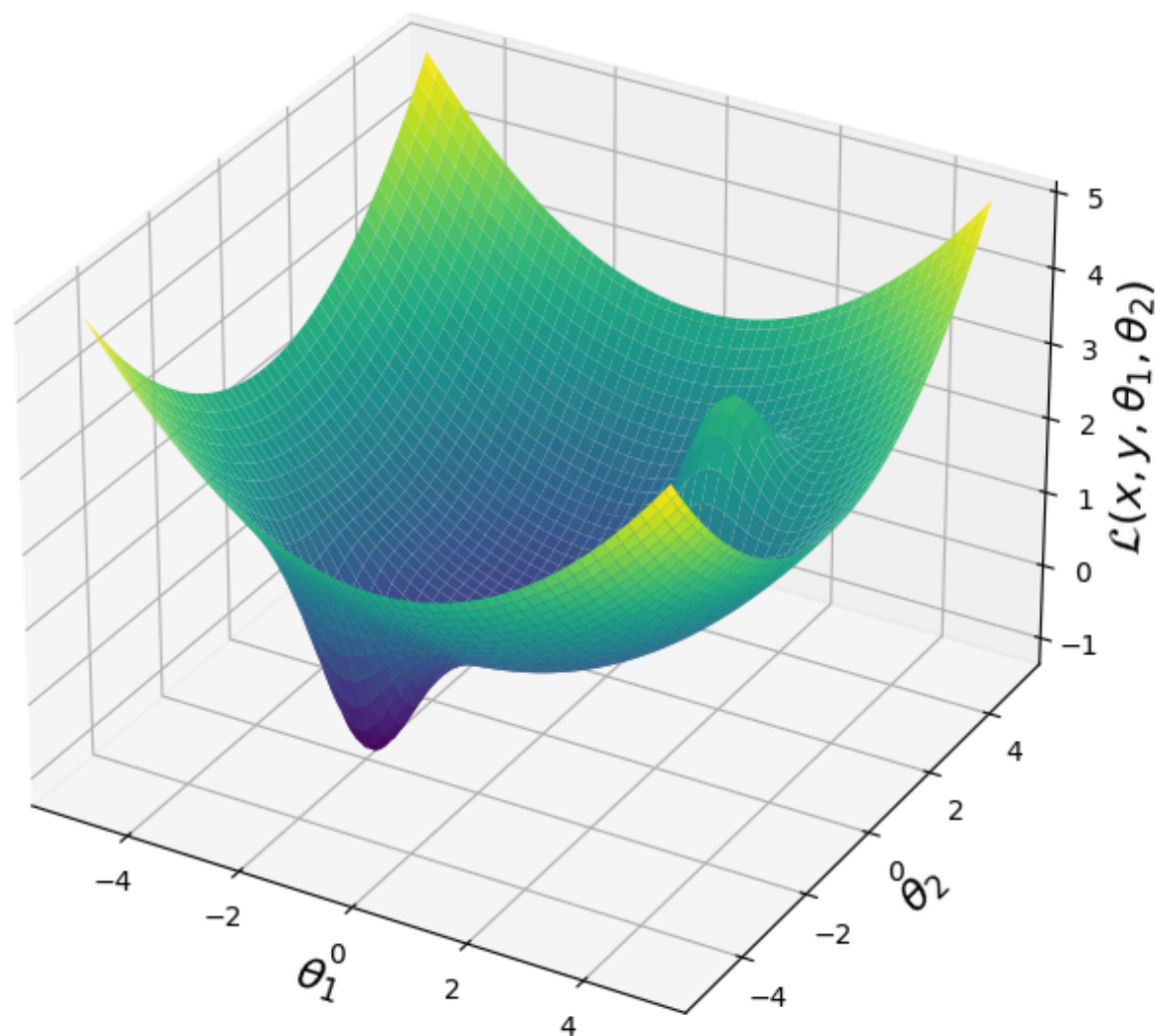
If we cannot compute the derivative/gradient, then we cannot know the direction to walk!

Gradient Descent

The gradient descent algorithm:

```
1: procedure GRADIENT DESCENT( $\mathbf{X}$ ,  $\mathbf{Y}$ ,  $\mathcal{L}$ ,  $t$ ,  $\alpha$ )
2:    $\boldsymbol{\theta} \leftarrow \text{Random}()$ 
3:   for  $i \in \{1 \dots t\}$  do
4:      $\triangleright$  Compute the gradient of the loss
5:      $\mathbf{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta})$ 
6:      $\triangleright$  Update the parameters using the negative gradient
7:      $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \mathbf{J}$ 
8:   end
9:   return  $\boldsymbol{\theta}$ 
10: end
```

Gradient Descent



Gradient Descent

Two main steps in gradient descent:

Step 1: Compute the gradient of the loss

Step 2: Update the parameters using the gradient

Let us start with step 1

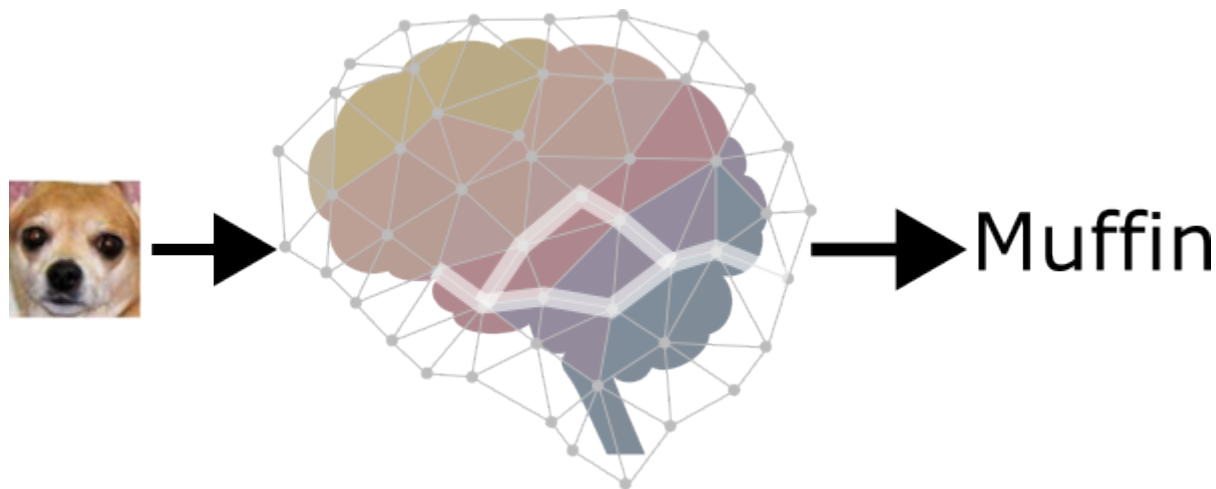
Backpropagation

Backpropagation

Goal: Compute the gradient of the loss $(\nabla_{\theta}\mathcal{L})(X, Y, \theta)$

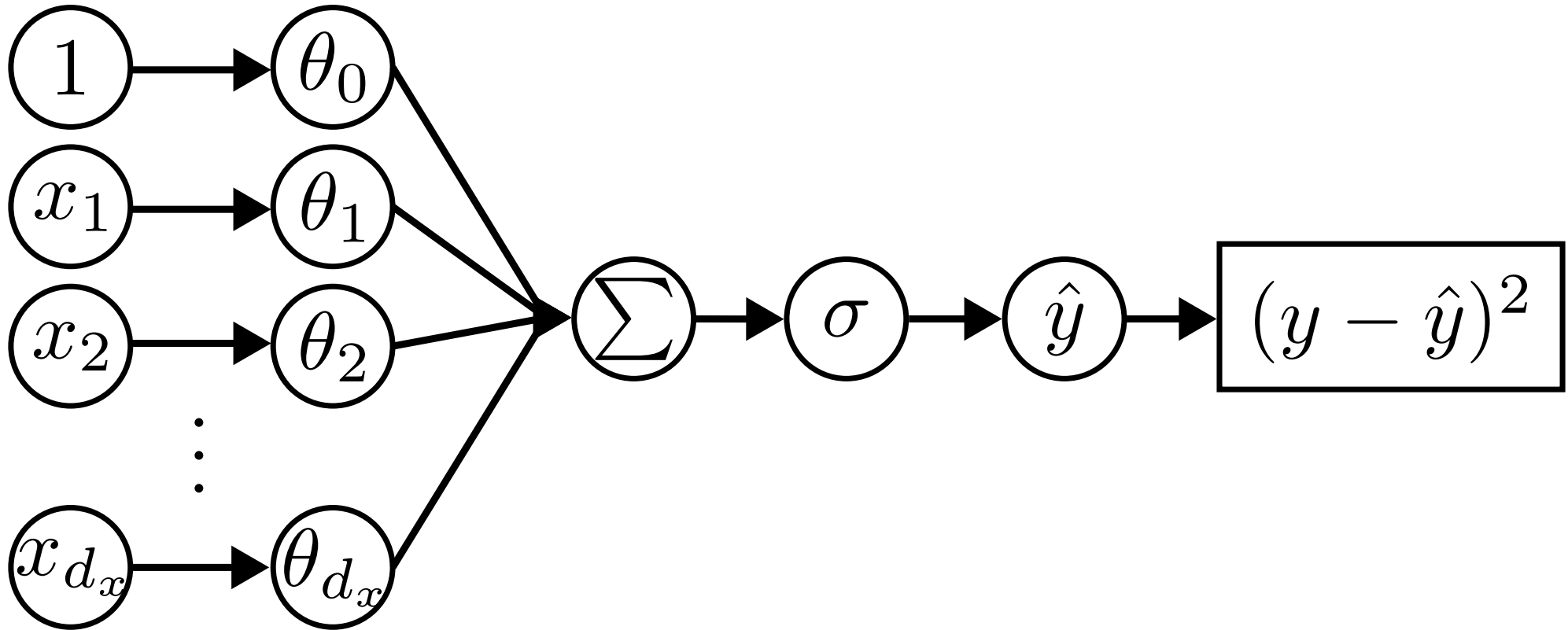
We call this process **backpropagation**

We propagate errors from the loss function **backward** through each layer of the neural network



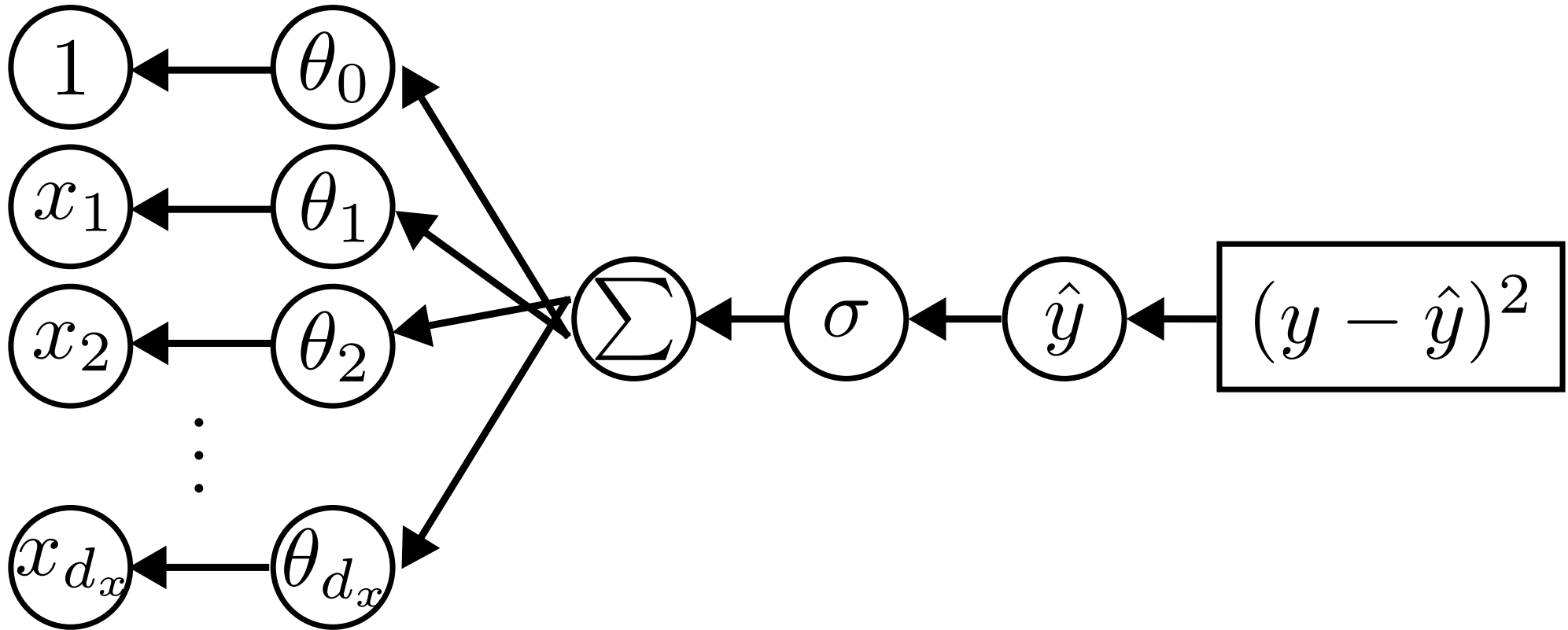
Backpropagation

Forward propagation



Backpropagation

Backward propagation



Backpropagation

Finding the gradient is necessary to use gradient descent!

First, we will find the gradient of a neural network layer

Then, we will find the gradient of a deep neural network

Finally, we will find the gradient of the loss function

Backpropagation

Start with the equation of a neural network layer

$$f(x, \theta) = \sigma(\theta^\top \bar{x})$$

Take the gradient of both sides

$$(\nabla_{\theta} f)(x, \theta) = \nabla_{\theta} [\sigma(\theta^\top \bar{x})]$$

$$\text{Chain: } \frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$$

$$(\nabla_{\theta} f)(x, \theta) = (\nabla \sigma)(\theta^\top \bar{x}) \cdot \nabla_{\theta} (\theta^\top \bar{x})$$

$$(\nabla_{\theta} f)(x, \theta) = (\nabla \sigma)(\theta^\top \bar{x}) \cdot \bar{x}$$

Backpropagation

$$(\nabla_{\theta} f)(x, \theta) = (\nabla \sigma)(\theta^{\top} \bar{x}) \cdot \bar{x}$$

The input

Gradient of activation evaluated at $\theta^{\top} \bar{x}$

This is the gradient of a neural network layer!

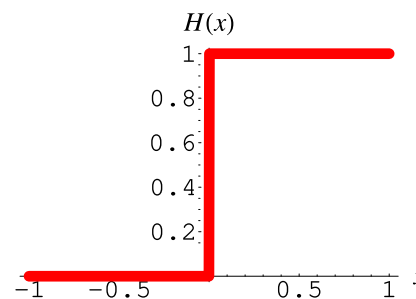
But what is $(\nabla \sigma)(\theta^{\top} \bar{x})$?

Backpropagation

$$(\nabla_{\theta} f)(x, \theta) = (\nabla \sigma)(\theta^{\top} \bar{x}) \cdot \bar{x}$$

Need to find $(\nabla \sigma)(z)$

Recall $\sigma(z)$



Question: What is $(\nabla \sigma)(z)$?

Answer: Does not exist! σ is not differentiable everywhere.

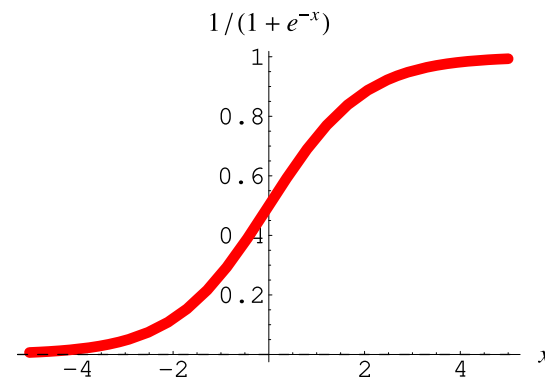
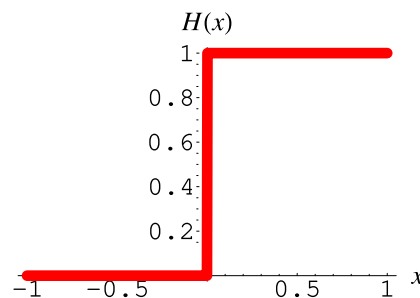
Even in closed interval, gradient is either ∞ or 0

Backpropagation

We use a differentiable approximation of the heaviside step function

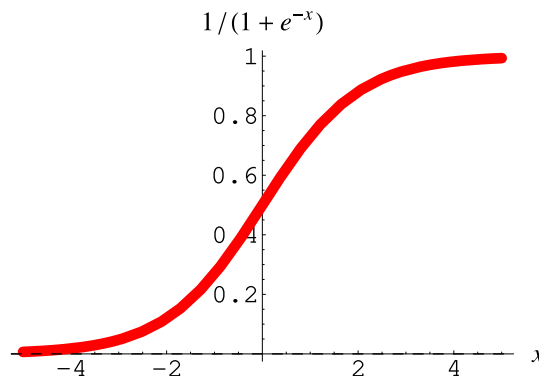
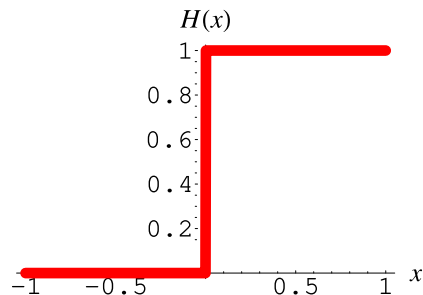
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

We call this approximation the **sigmoid function**



The sigmoid function has finite and nonzero derivative everywhere

Backpropagation



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The gradient of the sigmoid function is

$$(\nabla_z \sigma)(z) = \sigma(z) \cdot (1 - \sigma(z))$$

For vector $\mathbf{z}$:

$$(\nabla_{\mathbf{z}} \sigma)(\mathbf{z}) = \sigma(\mathbf{z}) \odot (1 - \sigma(\mathbf{z}))$$

Backpropagation

Back to our layer

$$(\nabla_{\theta} f)(x, \theta) = (\nabla \sigma)(\theta^{\top} \bar{x}) \cdot \bar{x}$$

Plug in the gradient of our new activation function

$$(\nabla \sigma)(z) = \sigma(z) \odot (1 - \sigma(z))$$

$$(\nabla_{\theta} f)(x, \theta) = [\sigma(\theta^{\top} \bar{x}) \odot (1 - \sigma(\theta^{\top} \bar{x}))] \odot \bar{x}$$

Backpropagation

$$(\nabla_{\boldsymbol{\theta}} f)(\boldsymbol{x}, \boldsymbol{\theta}) = [\sigma(\boldsymbol{\theta}^\top \bar{\boldsymbol{x}}) \odot (1 - \sigma(\boldsymbol{\theta}^\top \bar{\boldsymbol{x}}))] \odot \bar{\boldsymbol{x}}$$

This is the gradient for one layer in a neural network

We will use this to compute the gradient of a deep neural network

Backpropagation

Recall the deep neural network has many layers

$$f_1(\boldsymbol{x}, \boldsymbol{\varphi}) = \sigma(\boldsymbol{\varphi}^\top \overline{\boldsymbol{x}}) \quad f_2(\boldsymbol{x}, \boldsymbol{\psi}) = \sigma(\boldsymbol{\psi}^\top \overline{\boldsymbol{x}}) \quad \dots \quad f_\ell(\boldsymbol{x}, \boldsymbol{\xi}) = \sigma(\boldsymbol{\xi}^\top \overline{\boldsymbol{x}})$$

And that we call them in series

$$z_1 = f_1(\boldsymbol{x}, \boldsymbol{\varphi})$$

$$z_2 = f_2(z_1, \boldsymbol{\psi})$$

$$\vdots$$

$$z_\ell = f_\ell(z_{\ell-1}, \boldsymbol{\xi})$$

Backpropagation

Take the gradient of both sides

$$\begin{aligned}\nabla_{\varphi, \psi, \dots, \xi} z_1 &= (\nabla_{\varphi, \psi, \xi} f_1)(x, \varphi) \\ \nabla_{\varphi, \psi, \dots, \xi} z_2 &= (\nabla_{\varphi, \psi, \xi} f_2)(z_1, \psi) \\ &\vdots \\ \nabla_{\varphi, \psi, \dots, \xi} z_\ell &= (\nabla_{\varphi, \psi, \xi} f_\ell)(z_{\ell-1}, \xi)\end{aligned}$$

Each layer only uses one set of parameters

$$\begin{aligned}\nabla_{\varphi} z_1 &= (\nabla_{\varphi} f_1)(x, \varphi) \\ \nabla_{\psi} z_2 &= (\nabla_{\psi} f_2)(z_1, \psi) \\ &\vdots \\ \nabla_{\xi} z_\ell &= (\nabla_{\xi} f_\ell)(z_{\ell-1}, \xi)\end{aligned}$$

Backpropagation

The gradient of a deep neural network f is

$$(\nabla_{\theta} f)(\mathbf{x}, \theta) = (\nabla_{\varphi, \psi, \dots, \xi} f) \left(\mathbf{x}, \begin{bmatrix} \varphi \\ \psi \\ \vdots \\ \xi \end{bmatrix} \right) = \begin{bmatrix} (\nabla_{\varphi} f_1)(\mathbf{x}, \varphi) \\ (\nabla_{\psi} f_2)(\mathbf{z}_1, \psi) \\ \vdots \\ (\nabla_{\xi} f_{\ell})(\mathbf{z}_{\ell-1}, \xi) \end{bmatrix}$$

Where each layer gradient is

$$(\nabla_{\xi} f_{\ell})(\mathbf{z}_{\ell-1}, \xi) = (\sigma(\xi^{\top} \bar{\mathbf{z}}_{\ell-1}) \odot (1 - \sigma(\xi^{\top} \bar{\mathbf{z}}_{\ell-1}))) \bar{\mathbf{z}}_{\ell-1}$$

Backpropagation

We computed the gradient of a neural network layer

We computed the gradient of the neural network

Now, we must compute gradient of the loss function

Backpropagation

$$\mathcal{L}(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = \sum_{i=1}^n \left(f(\mathbf{x}_{[i]}, \boldsymbol{\theta}) - \mathbf{y}_{[i]} \right)^2$$

$$(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \left[\sum_{i=1}^n \left(f(\mathbf{x}_{[i]}, \boldsymbol{\theta}) - \mathbf{y}_{[i]} \right)^2 \right]$$

$$(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = \sum_{i=1}^n \nabla_{\boldsymbol{\theta}} \left[\left(f(\mathbf{x}_{[i]}, \boldsymbol{\theta}) - \mathbf{y}_{[i]} \right)^2 \right]$$

$$(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = \sum_{i=1}^n 2 \left(f(\mathbf{x}_{[i]}, \boldsymbol{\theta}) - \mathbf{y}_{[i]} \right) (\nabla_{\boldsymbol{\theta}} f)(\mathbf{x}_{[i]}, \boldsymbol{\theta})$$

Backpropagation

To summarize:

$$(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\mathbf{X}, \mathbf{Y}, \boldsymbol{\theta}) = \sum_{i=1}^n 2 \left(f(\mathbf{x}_{[i]}, \boldsymbol{\theta}) - \mathbf{y}_{[i]} \right) (\nabla_{\boldsymbol{\theta}} f)(\mathbf{x}_{[i]}, \boldsymbol{\theta})$$

$$(\nabla_{\boldsymbol{\theta}} f)(\mathbf{x}, \boldsymbol{\theta}) = (\nabla_{\varphi, \psi, \dots, \boldsymbol{\xi}} f) \left(\mathbf{x}, \begin{bmatrix} \varphi \\ \psi \\ \vdots \\ \boldsymbol{\xi} \end{bmatrix} \right) = \begin{bmatrix} (\nabla_{\varphi} f_1)(\mathbf{x}, \varphi) \\ (\nabla_{\psi} f_2)(\mathbf{z}_1, \psi) \\ \vdots \\ (\nabla_{\boldsymbol{\xi}} f_{\ell})(\mathbf{z}_{\ell-1}, \boldsymbol{\xi}) \end{bmatrix}$$

$$(\nabla_{\boldsymbol{\xi}} f_{\ell})(\mathbf{z}_{\ell-1}, \boldsymbol{\xi}) = [\sigma(\boldsymbol{\xi}^{\top} \bar{\mathbf{z}}_{\ell-1}) \odot (1 - \sigma(\boldsymbol{\xi}^{\top} \bar{\mathbf{z}}_{\ell-1}))] \bar{\mathbf{z}}_{\ell-1}$$

Backpropagation

Question: Why did we spend all this time deriving gradients?

Answer: The gradient is necessary for gradient descent

```
1: procedure GRADIENT DESCENT( $X, Y, \mathcal{L}, t, \alpha$ )
2:    $\theta \leftarrow \text{Random}()$ 
3:   for  $i \in \{1 \dots t\}$  do
4:      $\triangleright$  Compute the gradient of the loss
5:      $J \leftarrow (\nabla_{\theta} \mathcal{L})(X, Y, \theta)$ 
6:      $\triangleright$  Update the parameters using the negative gradient
7:      $\theta \leftarrow \theta - \alpha \cdot J$ 
8:   end
9:   return  $\theta$ 
10: end
```

Backpropagation

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \cdot (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}_t)$$

Coding

Coding

How do gradients work in torch?

The libraries compute the gradients using **autograd**

Autograd differentiates nested functions using the chain rule

$$(\nabla_{\theta} f(g(h)))(x, \theta) = (\nabla f)(g(h(x, \theta))) \cdot (\nabla g)(h(x, \theta)) \cdot (\nabla h)(x, \theta)$$

Engineers derived gradients for hundreds of functions $f, g, h, \dots$

Researchers derive their own analytical gradients like we did today

Now, let us look at torch optimization code

Coding

```
import torch
optimizer = torch.optim.SGD(lr=0.0001)

def L(model, X, Y):
    ...
    # Pytorch will record a graph of all operations
    # Everytime you do theta @ x, it stores inputs and outputs
    loss = L(X, Y, model) # compute gradient
    # Traverse the graph backward and compute the gradient
    loss.backward() # Sets .grad attribute on each parameter
    optimizer.step() # Update the parameters using .grad
    optimizer.zero_grad() # Set .grad to zero, DO NOT FORGET!!
```

Coding

Time for some interactive coding

https://colab.research.google.com/drive/1W8WVZ8n_9yJCcOqkPVURp_wJUx3EQc5w